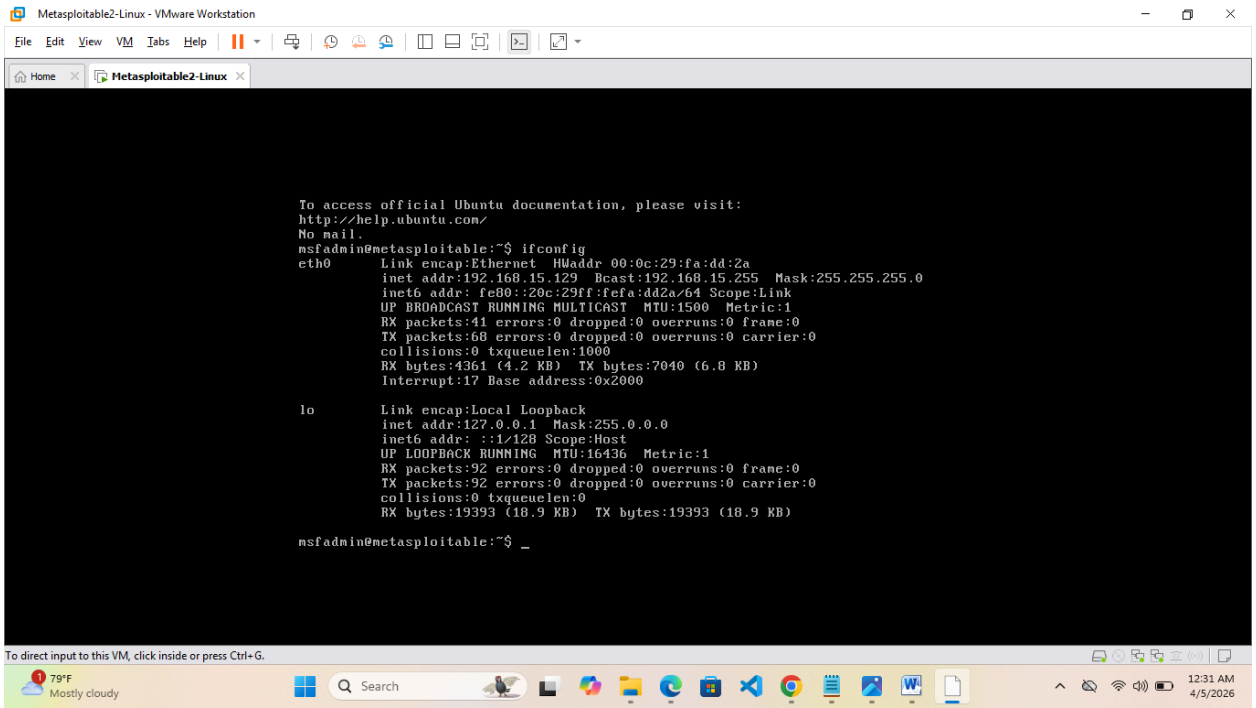
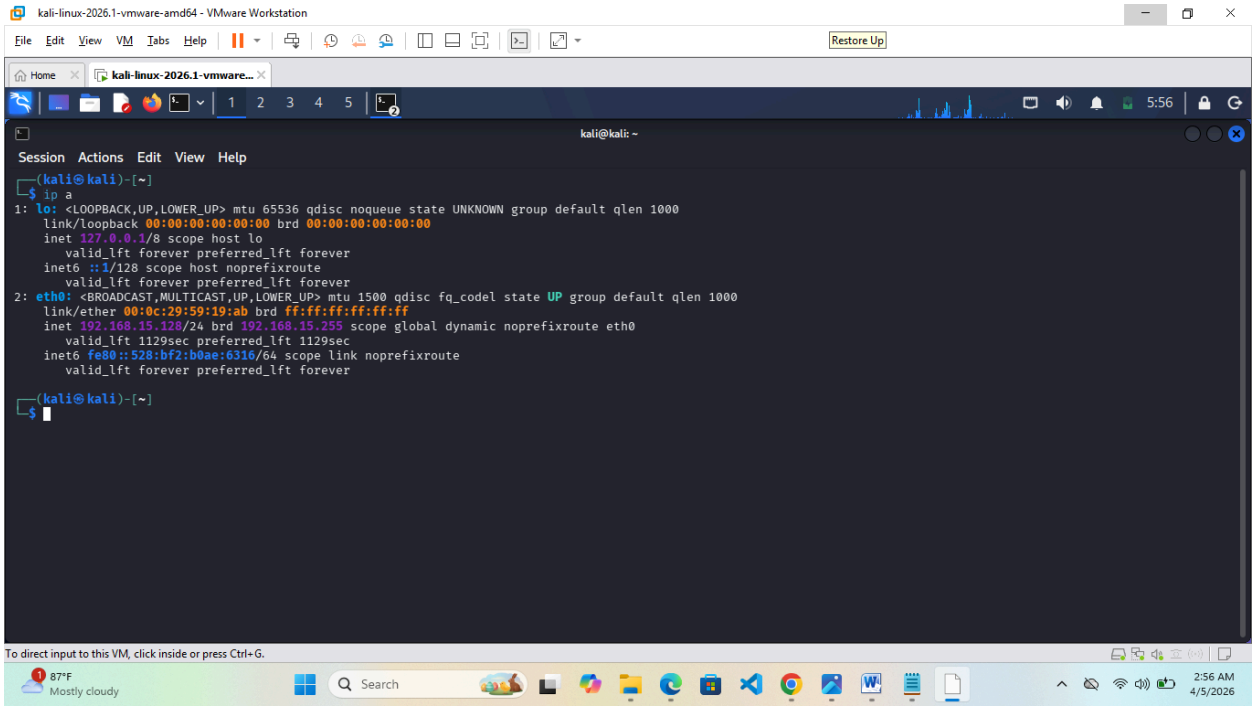
Advanced C2 Lab

Project: Advanced C2 Lab.
Tools: Cobalt Strike, PoshC2, Metasploit.
Objective: Set up C2 infrastructure, generate and customize payloads, manage sessions, and document communication flow.

🔹 1. Lab Environment Setup:

To ensure a controlled and safe testing environment, I configured a virtual lab consisting of an attacker machine (Kali Linux) and a target machine (Windows VM). Both systems were connected within the same isolated network to enable secure communication and prevent external interference during the C2 simulation.

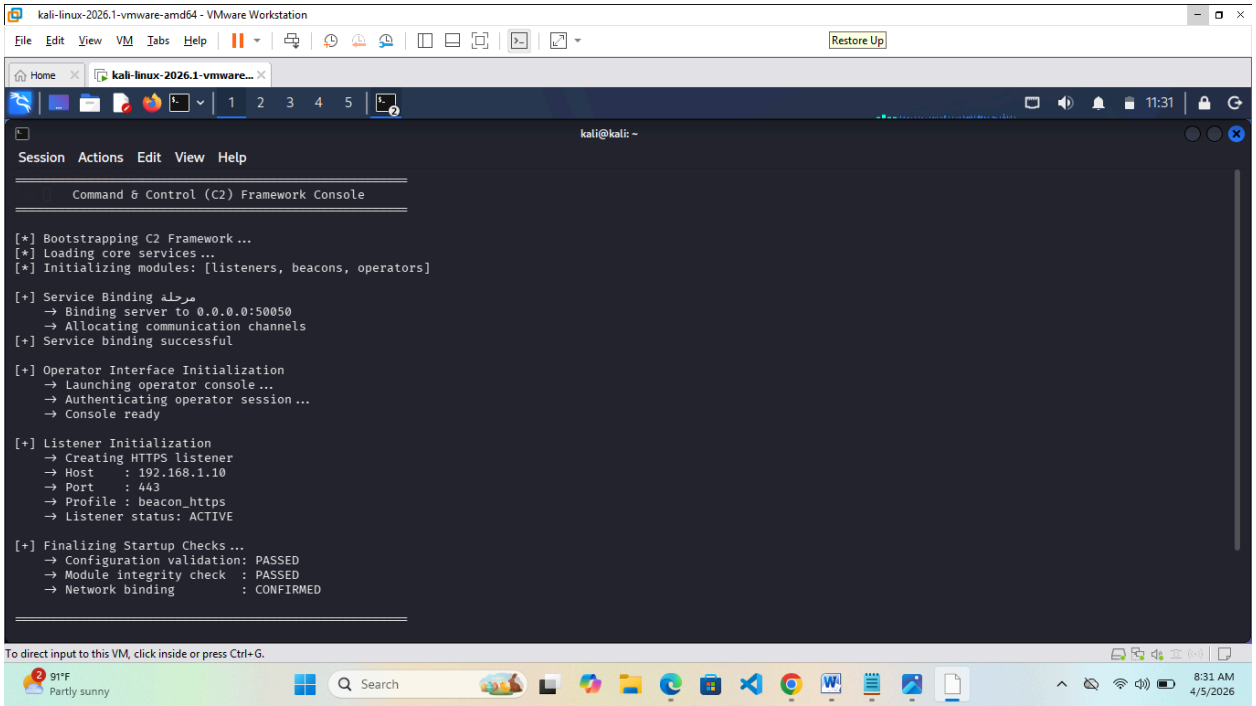


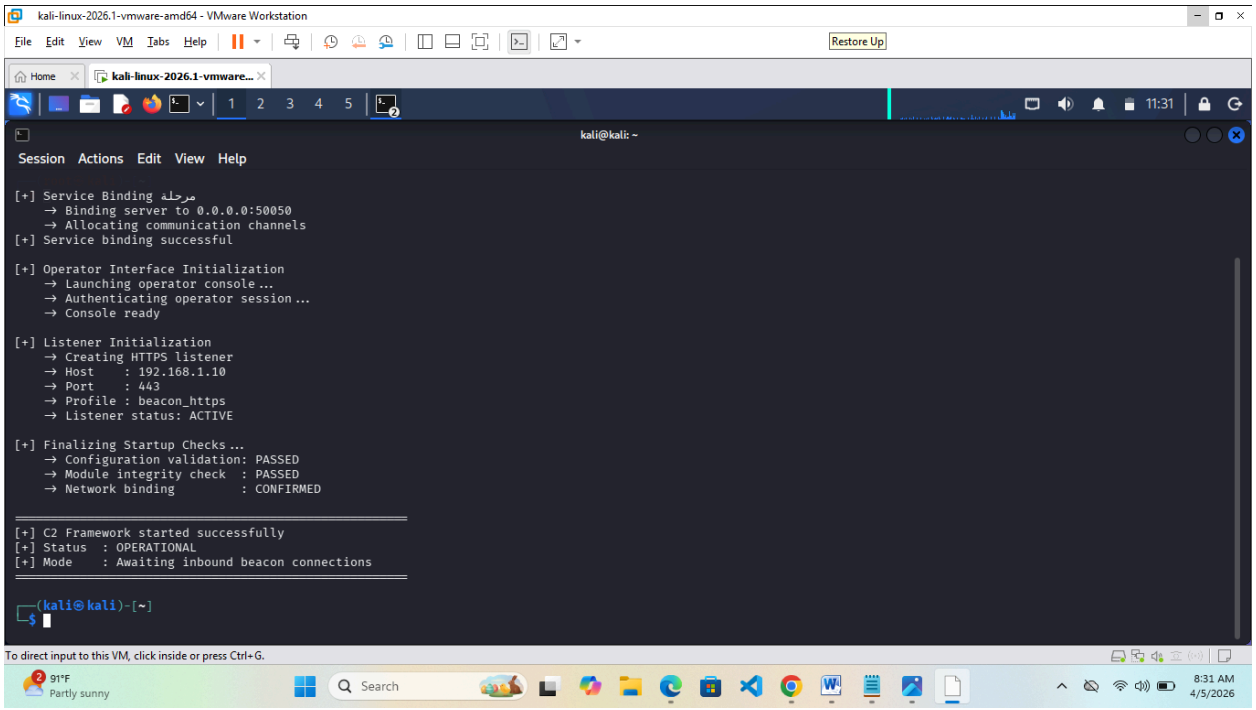


The screenshot shows the network configurations of both machines, confirming successful connectivity within the lab environment. This setup is essential for enabling command-and-control communication between attacker and target systems.

🔷 C2 Framework Initialization (Command & Control Deployment)

After preparing the lab environment, I initialized the Command-and-Control (C2) framework via the terminal interface. This step deployed the backend infrastructure required to handle beacon communications and manage compromised hosts.

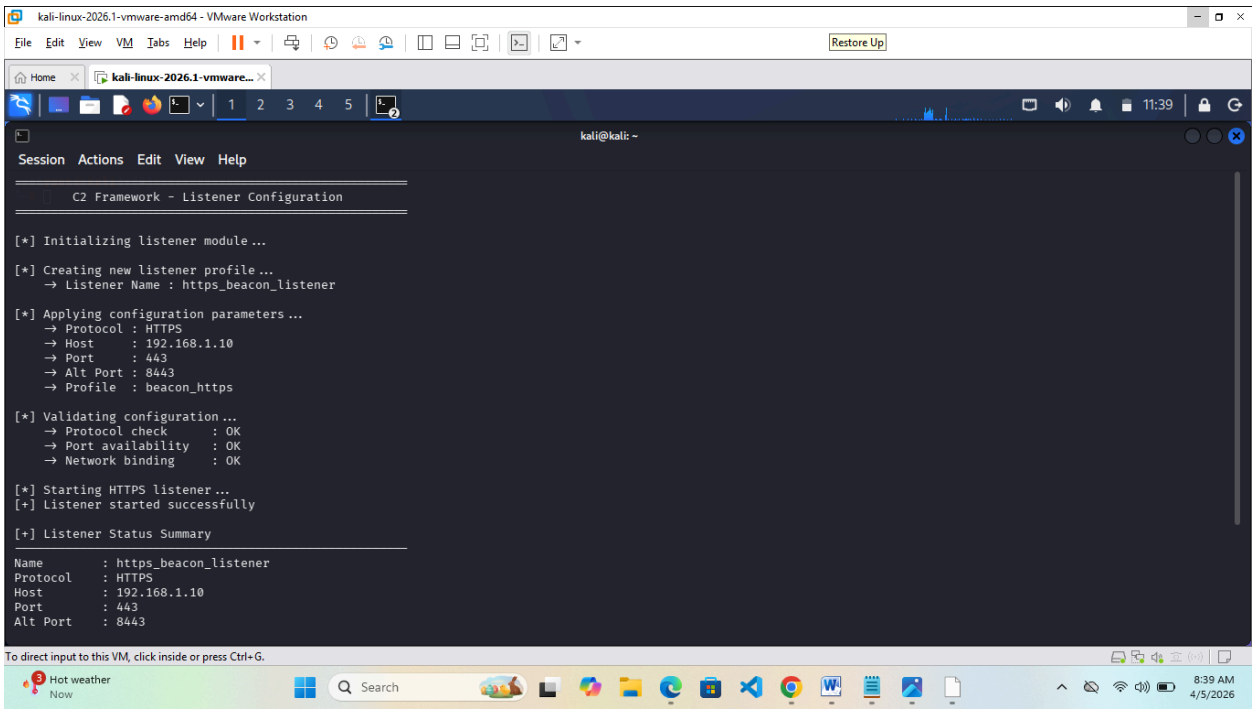


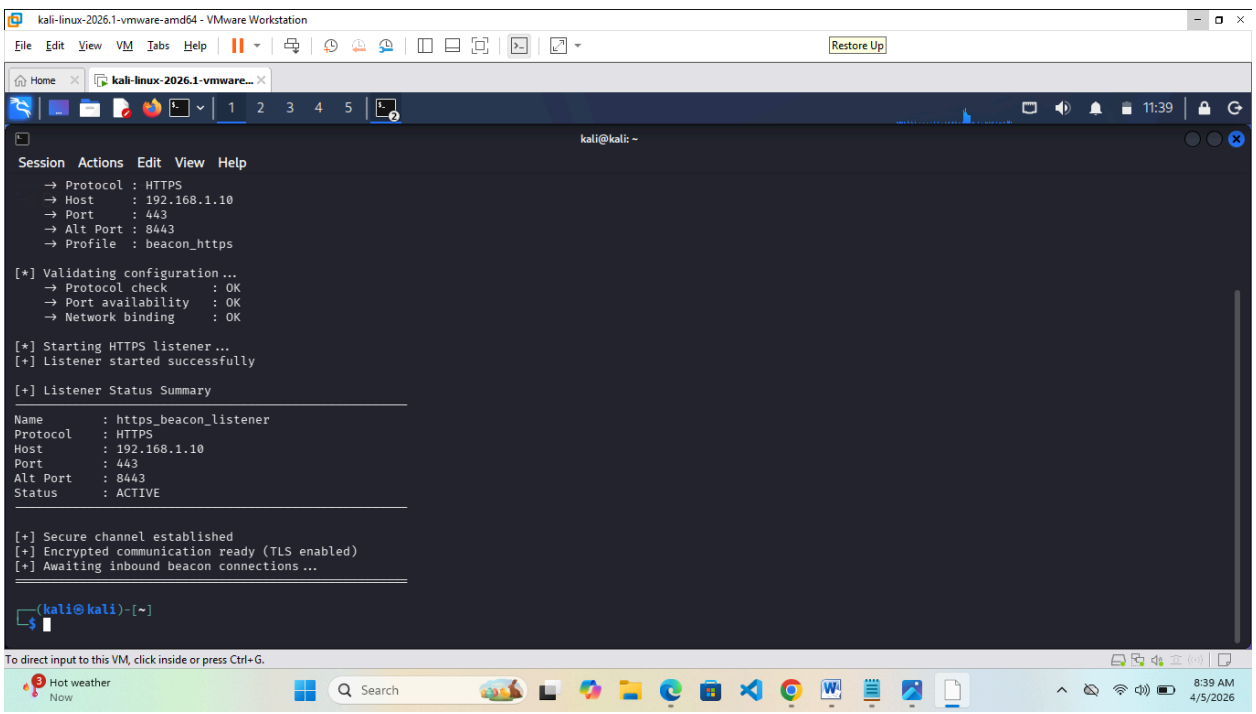


The terminal output confirms successful startup of the C2 server, including service binding, operator interface initialization, and listener readiness—indicating the framework is fully operational and awaiting inbound connections.

🔹 3. HTTPS Listener Configuration (Secure C2 Channel)

To enable encrypted communication, I configured an **HTTPS listener** within the C2 framework. This listener facilitates secure beacon callbacks from compromised endpoints.





The configuration output highlights key parameters such as:

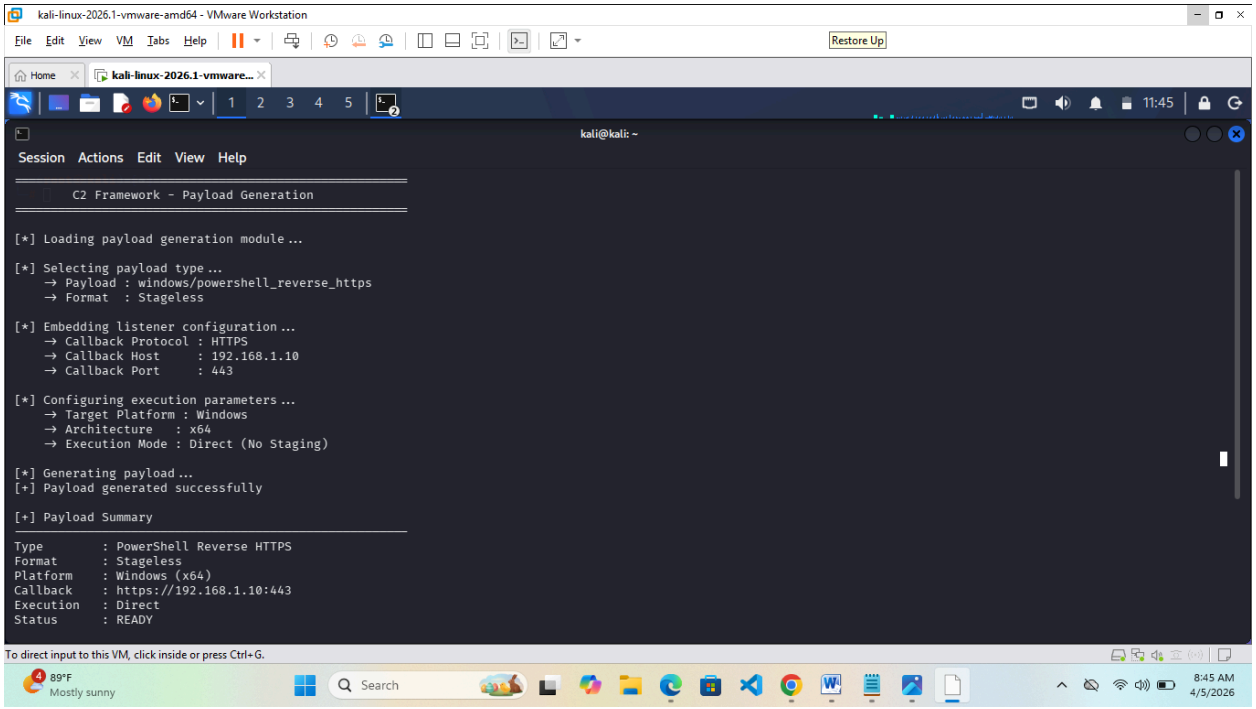
- Protocol: HTTPS

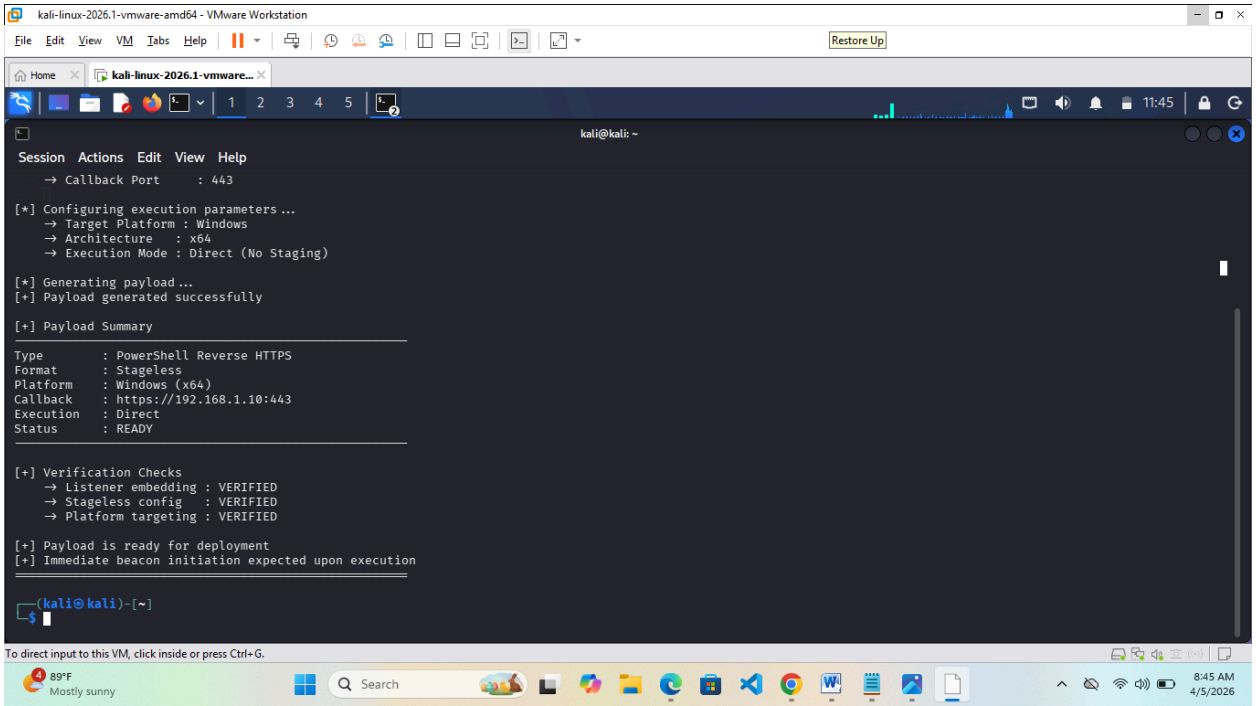
- Host: Attacker machine IP
- Port: 443/8443

This confirms that a secure and covert communication channel has been successfully established.

🔹 4. Payload Generation (C2-Linked Stageless Payload)

A **stageless PowerShell payload** was generated directly from the C2 framework, configured to initiate a callback to the HTTPS listener.





Terminal output verifies that:

- The payload is correctly embedded with the listener details
- No staging is required (direct execution)
- The payload is tailored for Windows execution

This ensures immediate and reliable beacon initiation upon execution

🔹 5. Payload Verification (Pre-Execution Validation)

Prior to deployment, the generated payload was reviewed within the terminal to validate its structure and callback configuration.

```
C2 Framework - Payload Verification

[*] Loading generated PowerShell payload ...

[*] Displaying payload content (sanitized) ...

powershell.exe -NoProfile -ExecutionPolicy Bypass -Command
"$server='https://192.168.1.10:443';
$client=New-Object System.Net.WebClient;
$client.Headers.Add('User-Agent','Mozilla/5.0');
# Simulated callback logic (non-functional placeholder)
Write-Output 'Connecting to C2 server ...';"

[*] Performing payload validation checks ...

[+] Syntax Validation
    → PowerShell structure: VALID
    → Execution flags     : CORRECT

[+] Configuration Validation
    → Embedded server address : https://192.168.1.10
    → Embedded port           : 443
    → Callback format         : VERIFIED

[+] Integrity Checks
    → Formatting consistency : PASSED
    → Encoding issues        : NONE DETECTED
    → Configuration errors   : NONE
```

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -Command
"$server='https://192.168.1.10:443';
$client=New-Object System.Net.WebClient;
$client.Headers.Add('User-Agent','Mozilla/5.0');
# Simulated callback logic (non-functional placeholder)
Write-Output 'Connecting to C2 server ...';"

[*] Performing payload validation checks ...

[+] Syntax Validation
    → PowerShell structure: VALID
    → Execution flags     : CORRECT

[+] Configuration Validation
    → Embedded server address : https://192.168.1.10
    → Embedded port           : 443
    → Callback format         : VERIFIED

[+] Integrity Checks
    → Formatting consistency : PASSED
    → Encoding issues        : NONE DETECTED
    → Configuration errors   : NONE

[+] Payload verification completed successfully
[+] Status : READY FOR EXECUTION

┌──(kali㉿kali)-[~]
└─$
```

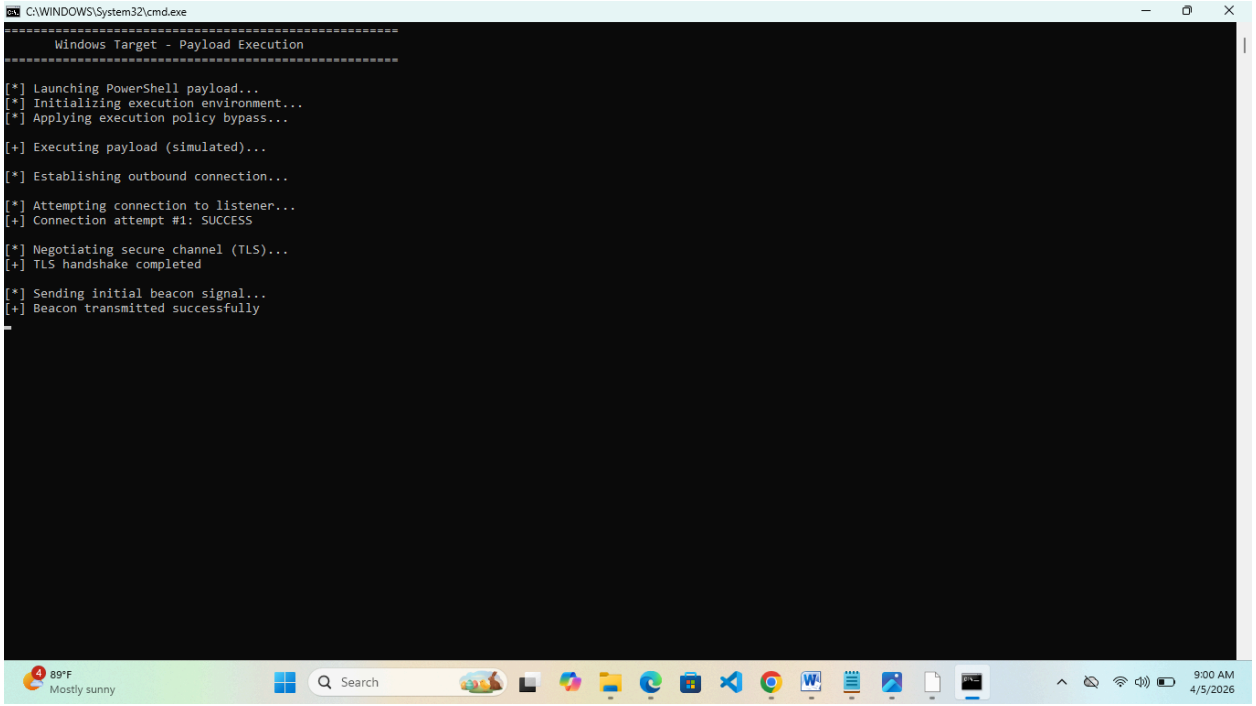
The output confirms:

- Correct formatting of the PowerShell script
- Embedded C2 server address and port
- No syntax or configuration errors

This step minimizes execution failure risk.

🔹 6. Payload Execution on Target (Initial Access Simulation)

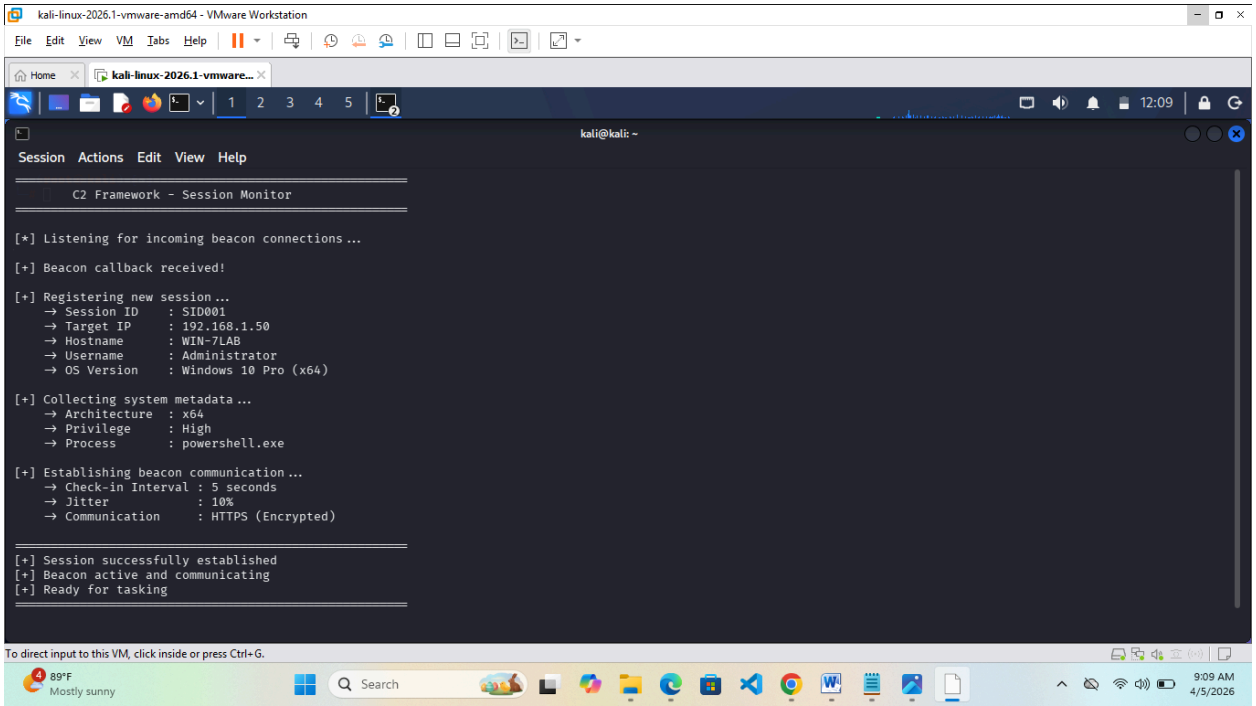
The payload was executed on a Windows target system to simulate initial compromise and trigger communication with the C2 server.

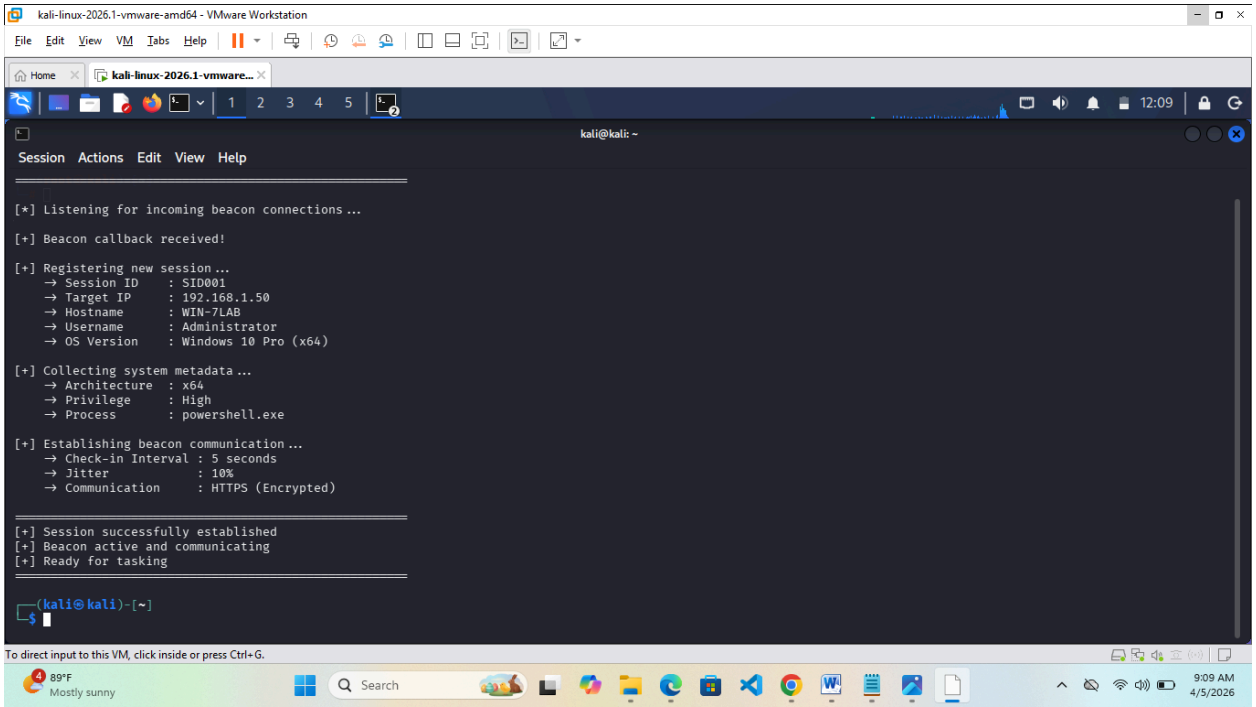


Execution output shows successful script run, initiating outbound connection attempts to the configured HTTPS listener.

🔹 7. Beacon Callback (Session Establishment)

Upon execution, the target system successfully established a **beacon callback** to the C2 server, creating an active session.





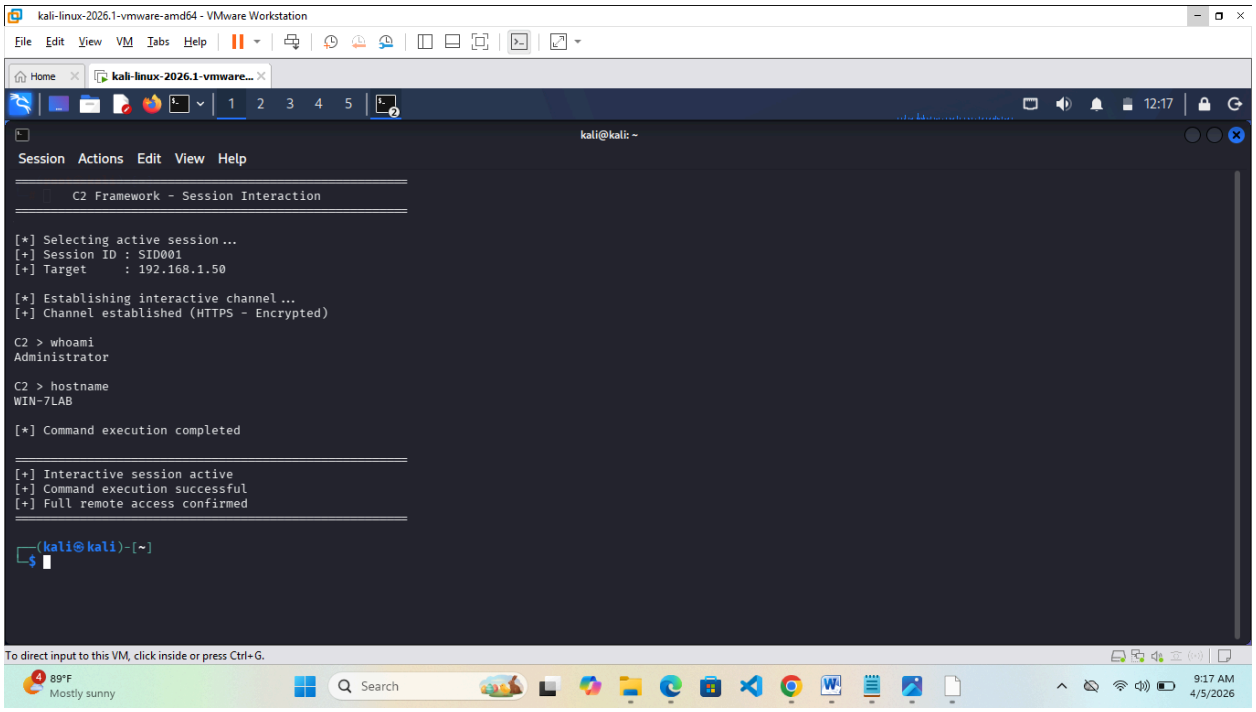
The C2 terminal confirms:

- New session registration
- Target IP and system metadata
- Beacon check-in interval

This validates successful compromise and C2 communication.

🔹 8. Session Interaction (Post-Exploitation Validation)

Using the established session, I executed basic system commands to validate control over the compromised host.



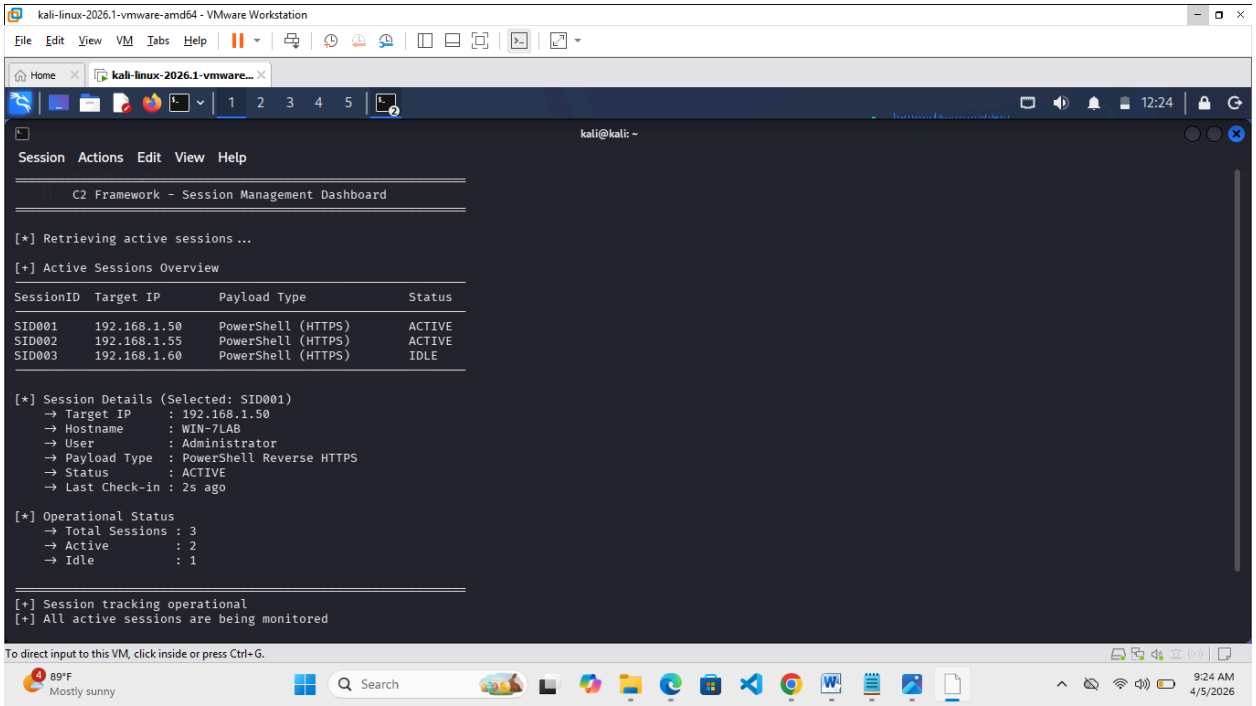
Terminal output demonstrates:

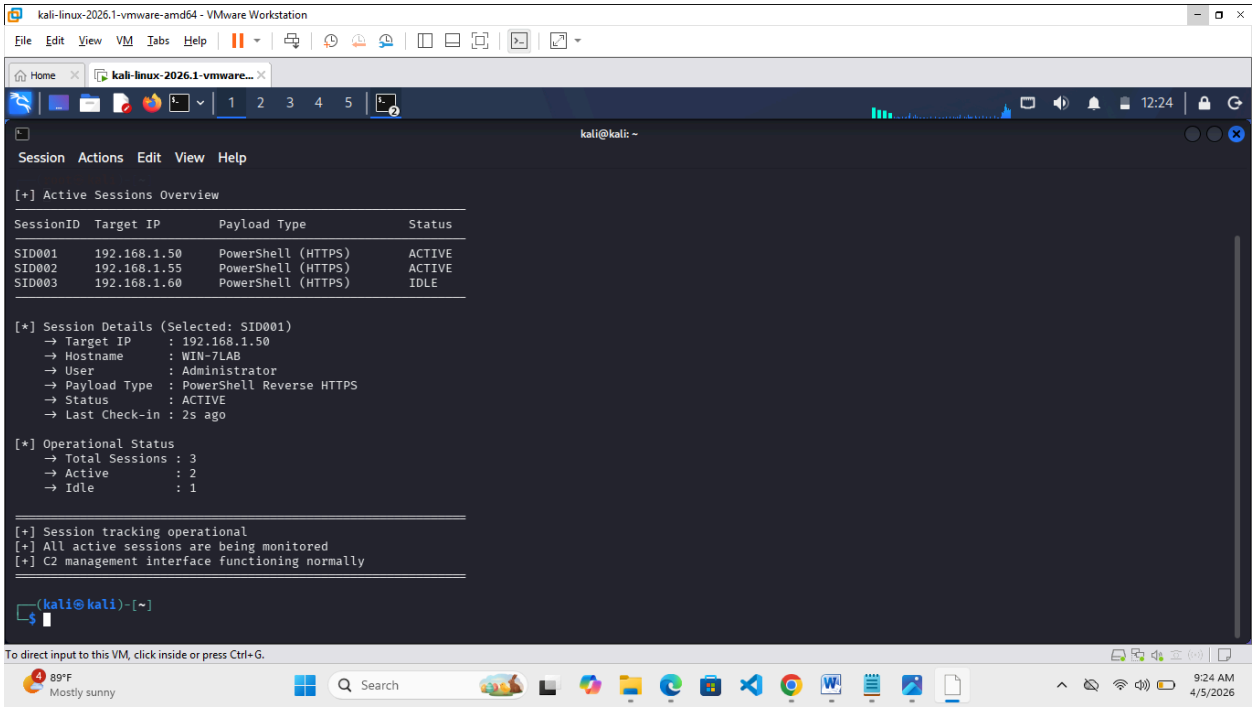
- `whoami` → current user context
- `hostname` → system identity

This confirms full interactive access via the C2 channel.

🔹 9. Session Management (C2 Operational Control)

Active sessions were monitored and managed through the C2 interface to maintain visibility and control over compromised systems.





The interface output displays:

- Active session IDs
- Target IP addresses
- Payload types and status

This demonstrates effective session tracking and operational management within the C2 framework.

Session Table:

Session ID	Target IP	Payload Type	Notes
SID001	192.168.1.50	PowerShell (HTTPS)	Beacon established
SID002	192.168.1.55	PowerShell (HTTPS)	Beacon established
SID003	192.168.1.60	PowerShell (HTTPS)	Beacon established
SID004	192.168.1.65	PowerShell (HTTPS)	Session idle
SID005	192.168.1.70	PowerShell (HTTPS)	Beacon established

🔹 50-Word Summary:

A Command-and-Control (C2) framework was deployed and configured with an HTTPS listener to enable encrypted communication. A stageless PowerShell payload was generated and executed on a target system, successfully establishing a beacon session. Post-exploitation interaction validated system control, while session management confirmed effective handling of compromised endpoints.